20. Python: File Handling

“Files help your programs remember even after they stop running.”

Previous Section:

</> 19. Python: Regular Expressions

Contents:

Imagine writing a program that completely forgets everything the moment it closes. No saved scores. No stored notes. No logs. No data persistence at all.

That would make programs feel temporary. This is where file handling becomes important.

Python allows programs to interact with files stored on your computer. Instead of keeping information only in memory while the program is running, we can save data into files and retrieve it later whenever needed.

Think about everyday applications:

- A text editor saving documents
- A game storing player progress
- A banking system recording transactions
- A chatbot remembering conversation logs

All of these rely on file handling.

In this section we focus mainly on working with **text files (.txt)** using Python. More advanced formats such as CSV, JSON, SQL databases, or Excel files are usually treated as separate topics because they introduce their own structures and libraries.

So before diving into advanced data systems, we first learn the foundation: How Python opens, reads, writes, and manages files.

And once you understand this properly, you'll realize something important: A program that can use files is no longer temporary. It becomes persistent.

1. Common File Handling Methods

Python provides several built-in methods for working with files.

These methods allow programs to:

- Create files

- Read existing data
- Write new information
- Append additional content
- Save results permanently

At the center of all file operations is the `open()` function.

Opening a File

Before reading or writing a file, Python must first open it.

Syntax:

```
open(filename, mode)
```

Example:

```
file = open("notes.txt", "r")
```

This opens `notes.txt` in **read mode**.

Creating a File

But of course we need the file to exist in the first place. If not, we create it:

```
with open("notes.txt", "w") as file:
    file.write("This is my first note.\n")
    file.write("Python makes file handling easy!\n")
```

Common File Modes

Mode	Meaning
`"r"`	Read a file
`"w"`	Write to a file (overwrites old content)

Mode	Meaning
`"a"`	Append new content
`"rb"`	Read binary files
`"wb"`	Write binary files

Reading an Entire File

The `read()` method retrieves the entire content of a file. Example:

```
file = open("notes.txt", "r")

content = file.read()
print(content)

file.close()
```

```
Python is fun
File handling is important
```

The output assumes the `notes.txt` file contains such content

Reading One Line at a Time

Sometimes reading the entire file is unnecessary. `readline()` reads a single line. Example:

```
file = open("notes.txt", "r")

line1 = file.readline()
line2 = file.readline()

print(line1)
print(line2)

file.close()
```

```
Python is fun
File handling is important
```

This is useful for processing files gradually instead of loading everything at once.

Reading All Lines into a List

The `readlines()` method stores all lines into a Python list. Example:

```
file = open("notes.txt", "r")

lines = file.readlines()
print(lines)

file.close()
```

```
['Python is fun\n', 'File handling is important']
```

Notice that newline characters (`\n`) are preserved.

Writing to a File

The `write()` method stores text into a file. Example:

```
file = open("message.txt", "w")
file.write("Hello World")
file.close()
```

If `message.txt` does not exist, Python creates it automatically. If it already exists, `"w"` mode replaces the old content completely.

Appending Content

To add content without deleting existing data, use append mode (`"a"`). Example:

```
file = open("message.txt", "a")
file.write("\nNew line added.")
file.close()
```

Result inside the file:
Hello World
New line added.

Writing Multiple Lines

The `writelines()` method writes multiple strings from a list. Example:

```
file = open("students.txt", "w")

names = ["Alice\n", "Bob\n", "Charlie\n"]

file.writelines(names)
file.close()
```

Alice
Bob
Charlie

Closing a File

The `close()` method releases the file resource. Example:

```
file = open("notes.txt", "r")

print(file.read())
file.close()
```

Failing to close files properly may cause memory issues, data corruption, locked files

Using `with open()`

Python provides a cleaner and safer approach:

```
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)
```

Once the block ends, Python automatically closes the file. This is considered the modern and recommended way to handle files.

A Simple Complete Example

```
with open("diary.txt", "w") as file:
    file.write("Today I learned Python file handling.")

with open("diary.txt", "r") as file:
    print(file.read())
```

```
Today I learned Python file handling.
```

This small example demonstrates the full cycle:

> ***Create a file → Write data → Save it → Reopen it → Read the stored information back into the program***

2. File Handling with Conditional Statements or Loops

File handling becomes much more useful when combined with conditional statements and loops. Because in real-world programs, we rarely just open a file and print everything blindly.

We usually want to:

- Check conditions
- Process lines differently
- Search for information
- Count occurrences
- Filter specific content
- Repeat operations automatically

This is where loops and conditional statements become extremely important.

Basic File Reading Example

```
# Open a file in write mode and write "Python"
with open("example.txt", "w") as file:
    file.write("Python")

# Open the file in read mode and print its content
with open("example.txt", "r") as file:
    content = file.read()
    print("File content:", content)
```

File content: Python

Creating a Multi-Line File

```
# Create a sample file with multiple lines
with open("example2.txt", "w") as file:
    file.write("Python\nHigh-Level\nInterpreted\nObject-Ori
ented\nProgramming Language\n")
```

This creates a text file containing multiple lines.

Using `readline()`

The `readline()` method retrieves one line at a time.

```
# Using readline() to read one line at a time
with open("example2.txt", "r") as file:
    print("Using readline():")

    line1 = file.readline()
    line2 = file.readline()

    print(line1.strip())
    print(line2.strip())
```

```
Using readline():
Python
High-Level
```

The `strip()` method removes extra newline characters.

Using `readlines()` with a Loop

```python
# Using readlines() to read all lines at once
with open("example2.txt", "r") as file:
    print("\nUsing readlines():")

    lines = file.readlines()

    for line in lines:
        print(line.strip())
```

```
Using readlines():
Python
High-Level
Interpreted
Object-Oriented
Programming Language
```

The `for` loop processes each line automatically.

File Handling with Conditional Statements

Conditional statements allow programs to make decisions while reading files.

- Example: Detecting a Specific Word

```python
with open("example2.txt", "r") as file:

    for line in file:

        if "Python" in line:
            print("Python found!")
        else:
            print("Other text:", line.strip())
```

```
Python found!
Other text: High-Level
Other text: Interpreted
Other text: Object-Oriented
Other text: Programming Language
```

The program checks each line individually using an `if-else` statement.

- Example: Counting Total Lines

```
count = 0

with open("example2.txt", "r") as file:
    for line in file:
        count += 1

print("Total lines:", count)
```

Total lines: 5

This demonstrates how loops can process files repeatedly.

- Example: Counting a Word Occurrence

```
count = 0

with open("example2.txt", "r") as file:
    for line in file:
        if "Programming" in line:
            count += 1

print("Occurrences found:", count)
```

Occurrences found: 1

- Example: Writing Numbers Using a Loop

Loops are also useful when writing repetitive content into files.

```python
with open("numbers.txt", "w") as file:

    for i in range(1, 6):
        file.write(f"Number {i}\n")
```

Content inside `numbers.txt`:

```
Number 1
Number 2
Number 3
Number 4
Number 5
```

- Example: Filtering Short Words

```python
with open("example2.txt", "r") as file:
    for line in file:
        word = line.strip()

        if len(word) > 10:
            print(word)
```

```
Interpreted
Object-Oriented
Programming Language
```

This combines: File handling; Loops; Conditional statements; String operations. All together.

Summary

File handling becomes far more powerful when combined with loops and conditional statements.

Instead of simply reading or writing files, programs can:

- Search for specific information
- Process data automatically

- Count occurrences
- Filter content
- Generate repeated outputs
- Analyze text dynamically

This is why file handling is heavily used in:

- Data analysis
- Log processing
- User management systems
- Banking systems
- AI datasets
- Text processing applications

Once loops and conditions are integrated into file handling, Python programs begin to feel far more intelligent and automated.

References

https://www.geeksforgeeks.org/python/file-handling-python/

https://pynative.com/python/file-handling/

Next Section:

</> 21. Python: Exception Handling